TUGAS

SISTEM TERDISTRIBUSI DAN PARALEL A

Distributed Synchronization System

LAPORAN TUGAS INDIVIDU
MATA KULIAH SISTEM TERDISTRIBUSI DAN PARALEL A

Disusun oleh:
Nama : Risky Alfienda NIM : 11231087

Program Studi : Informatika

Fakultas Sains dan Teknologi
Institut Teknologi Kalimantan 2026

1. Arsitektur Sistem

Sistem ini merupakan implementasi distributed synchronization system berbasis microservice nodes yang berjalan di Docker Compose. Setiap komponen dijalankan sebagai service FastAPI terpisah dan berkomunikasi melalui HTTP asynchronous. Redis digunakan sebagai penyimpanan state terdistribusi untuk queue, metadata, dan persistence.

Arsitektur terdiri atas beberapa kelompok node:

1. Lock Manager: lock_a, lock_b, dan lock_c.
2. Distributed Queue: queue_a, queue_b, dan queue_c.
3. Distributed Cache: cache_a, cache_b, dan cache_c.
4. PBFT Nodes: pbft_a, pbft_b, dan pbft_c.
5. Gateway: service untuk geo-routing dan adaptive balancer.
6. Redis Server: penyimpanan state dan persistence.

```
Host Machine
  |
  +-- Docker network: distnetv2
      |
      +-- Redis:6379
      |
      +-- Lock Manager
      |   +-- lock_a:9000
      |   +-- lock_b:9001
      |   +-- lock_c:9002
      |
      +-- Distributed Queue
      |   +-- queue_a:9100
      |   +-- queue_b:9101
      |   +-- queue_c:9102
      |
      +-- Distributed Cache
      |   +-- cache_a:9200
      |   +-- cache_b:9201
      |   +-- cache_c:9202
      |
      +-- Gateway:9300
      |
      +-- PBFT
          +-- pbft_a:9400
          +-- pbft_b:9401
          +-- pbft_c:9402
```

1.1 Arsitektur Diagram

Deskripsi singkat komponen:

- Lock Manager menangani shared lock, exclusive lock, antrean lock, deadlock detection, dan replikasi operasi lock menggunakan Raft.
- Distributed Queue menangani publish, consume, ack, consistent hashing, replica owner, visibility timeout, dan at-least-once delivery.
- Distributed Cache menangani cache coherence MESI-style, TTL, invalidation, update propagation, LRU replacement, dan metrics.
- Redis menyimpan pesan queue, metadata owner, state ready/inflight, dan data yang perlu bertahan setelah node restart.
- Gateway menyediakan simulasi multi-region routing dan adaptive load balancing.
- PBFT menyediakan implementasi bonus consensus untuk request Byzantine-tolerant secara dasar.

Kelebihan arsitektur ini adalah modular dan mudah di-scale. Node dapat dikembangkan secara terpisah sesuai role masing-masing. Dengan 3 node pada komponen utama, sistem dapat mensimulasikan quorum, replikasi, dan fault tolerance dasar.

2. Penjelasan Algoritma yang Digunakan

2.1 Consistent Hashing

Consistent hashing digunakan pada Distributed Queue. Kombinasi topic dan key di-hash ke ring virtual untuk menentukan owner node.

Manfaat consistent hashing:

- Distribusi pesan lebih merata.

- Penambahan atau pengurangan node hanya memindahkan sebagian kecil key.

- Owner message dapat dihitung ulang tanpa koordinasi global.

- Queue dapat mendukung producer dan consumer dari beberapa node.

2.2 Raft Consensus Algorithm

Raft digunakan pada Distributed Lock Manager untuk menjaga konsistensi operasi lock. Setiap operasi penting seperti acquire dan release direpresentasikan sebagai command yang direplikasi ke node lain.

Fase utama Raft:

1. Leader Election

Node dapat menjadi follower, candidate, atau leader. Jika follower tidak menerima heartbeat dalam batas waktu election timeout, node dapat menjadi candidate dan meminta vote ke node lain.

2. Log Replication

Leader menerima request write dan mengirim command ke follower melalui append entries. Command lock disimpan sebagai log entry sebelum diterapkan ke state machine.

3. Commit and Apply

Command dianggap committed jika mayoritas node menerima append entries. Setelah itu command diterapkan ke state machine lock.

Keunggulan Raft :

- Menghindari split-brain melalui term dan voting.
- Memastikan operasi lock diterapkan dengan urutan yang konsisten.
- Menyediakan dasar fault tolerance dengan quorum 3 node.
- Membatasi write agar diproses melalui leader.

2.3 At-least-once Delivery Guarantee

Queue menggunakan visibility timeout untuk menjamin at-least-once delivery. Ketika consumer mengambil message, message dipindahkan dari ready queue ke inflight queue. Jika consumer tidak melakukan ack sebelum deadline, reaper akan mengembalikan message ke ready queue.

Konsekuensi dari pendekatan ini:

- Message tidak hilang ketika consumer gagal.
- Message mungkin terkirim lebih dari sekali.
- Consumer perlu idempotent jika dipakai pada sistem produksi.

2.4 MESI-style Cache Coherence

Distributed Cache menggunakan pendekatan MESI-style coherence. Tujuannya adalah menjaga agar update pada satu cache node tidak membuat node lain membaca data lama.

State utama:

- M (Modified): data dimodifikasi pada satu node.

- E (Exclusive): data valid hanya pada satu node.

- S (Shared): data dapat dibaca oleh beberapa node.

- I (Invalid): data tidak valid atau sudah dihapus.

Ketika POST /cache/put dilakukan, node penerima memperbarui state lokal dan mengirim invalidation/update ke peer. Cache juga mendukung TTL dan LRU replacement agar memory usage tetap terkendali.

2.4 Algoritma Bonus

PBFT diimplementasikan sebagai fitur bonus. Node menerima request, membuat digest, melakukan prepare, commit,dan mencatat decided request saat quorum terpenuhi.

Geo-routing memakai peta latency antar region untuk memilih region tujuan. Adaptive balancer menyimpan laporan latency dan status node, lalu memilih node dengan performa terbaik.

3. API Documentation (Swagger/OpenAPI)

FastAPI secara otomatis menyediakan Swagger UI pada setiap service melalui path /docs. Selain itu, spesifikasi OpenAPI ringkas sudah disediakan pada: file *docs/api_spec.yaml*

Endpoint utama:

Komponen	Endpoint	Fungsi
System	GET /health	Health check
System	GET /readyz	Readiness check
System	GET /metrics	Prometheus metrics
Lock	POST /lock/acquire	Acquire shared/exclusive lock
Lock	POST /lock/release	Release lock
Lock	POST /lock/deadlock	Cek deadlock
Lock	GET /lock/state	Melihat state lock
Lock	GET /raft/role	Melihat role Raft
Queue	POST /queue/publish	Publish message
Queue	POST /queue/consume	Consume message
Queue	POST /queue/ack	Ack message
Queue	POST /queue/ack_owner	Ack message berdasarkan owner
Queue	GET /queue/owners	Melihat owner hash ring
Cache	POST /cache/put	Menulis cache
Cache	GET /cache/get	Membaca cache
Cache	GET /cache/state	Melihat state cache

Gambar 3.1 Tabel Endpoint 0.1

Komponen	Endpoint	Fungsi
Cache	GET /cache/metrics	Melihat metrics cache
PBFT	POST /pbft/request	Submit request PBFT
PBFT	GET /pbft/status	Melihat status PBFT
Gateway	GET /geo/route	Latency-aware routing
Gateway	POST /balancer/report	Mengirim laporan latency
Gateway	GET /balancer/next	Memilih node terbaik

Gambar 3.2 Tabel Endpoint 0.2

Contoh payload acquire lock:

```
{
"resource": "r1",
"mode": "exclusive",
"client_id": "c1",
"timeout_ms": 2000
}
```

Contoh payload payload cache put:

```
{
"key": "k1",
"value": "v1"
}
```

4. Deployment Guide & Troubleshooting

Deployment sistem dilakukan untuk memastikan seluruh komponen distributed synchronization system dapat berjalan secara terintegrasi dalam satu lingkungan eksekusi yang konsisten. Pada implementasi ini, proses deployment memanfaatkan Docker Compose agar setiap node dapat dijalankan sebagai container terpisah namun tetap berada dalam satu jaringan virtual yang sama. Pendekatan ini mempermudah simulasi arsitektur distributed system secara lokal tanpa memerlukan banyak mesin fisik.

Prasyarat yang harus terpenuhi sebelum menjalankan sistem:

- Docker Desktop
- Docker Compose.
- Python 3.10+.
- PowerShell.

4.1 Langkah deployment:

1. Install dependency lokal

Sebelum deployment dilakukan, seluruh dependency Python harus dipasang pada lingkungan lokal. Langkah ini bertujuan agar proses pengujian, benchmarking, dan utilitas tambahan dapat berjalan dengan baik di luar container. Instalasi dilakukan menggunakan file `requirements.txt` yang berisi seluruh pustaka yang dibutuhkan proyek.

```
python -m pip install -r .\requirements.txt
```

2. Build dan jalankan seluruh container.

Setelah dependency tersedia, tahap berikutnya adalah membangun image container dan menjalankan seluruh service menggunakan Docker Compose. Perintah ini akan membaca konfigurasi pada file docker-compose.yml, kemudian membuat container untuk setiap node seperti lock manager, queue system, cache node, gateway, PBFT node, serta Redis sebagai penyimpanan state**.**

```
docker compose -f .\docker\docker-compose.yml up -d --build
```

Parameter -d digunakan agar container berjalan di background, sedangkan --build memastikan image dibangun ulang jika terdapat perubahan source code. Dengan demikian, seluruh komponen akan aktif secara bersamaan sebagai simulasi cluster terdistribusi.

3. Cek Container

Setelah proses build selesai, kondisi container perlu diverifikasi untuk memastikan semua service telah aktif. Pemeriksaan dilakukan menggunakan perintah berikut:

```
docker compose -f .\docker\docker-compose.yml ps
```

Output dari perintah tersebut akan menampilkan daftar container beserta statusnya. Container dengan status Up menandakan service berhasil dijalankan.

4. Cek readiness service utama

Tahap selanjutnya adalah melakukan readiness check pada service utama. Berbeda dengan health check biasa, readiness check memastikan bahwa service benar-benar siap menerima request, termasuk koneksi internal ke dependency seperti Redis dan komunikasi antar-node.

```
curl.exe -s http://127.0.0.1:9000/readyz

curl.exe -s http://127.0.0.1:9100/readyz

curl.exe -s http://127.0.0.1:9200/readyz

curl.exe -s http://127.0.0.1:9300/readyz

curl.exe -s http://127.0.0.1:9400/readyz
```

5. Jalankan test

Setelah seluruh node berada pada kondisi siap, pengujian integrasi dapat dijalankan untuk memverifikasi bahwa seluruh fitur utama bekerja sesuai spesifikasi. Pengujian ini mencakup lock acquisition, queue publish-consume, cache coherence, hingga komunikasi antar-node.

```
.\scripts\run_tests.ps1 -SkipDocker
```

Parameter -SkipDocker digunakan karena container telah dijalankan sebelumnya, sehingga skrip hanya fokus pada proses testing.

6. Jalankan benchmark Locust

Untuk mengevaluasi performa sistem di bawah beban multi-user, benchmark dilakukan menggunakan Locust. Simulasi ini merepresentasikan beberapa user yang secara bersamaan mengakses endpoint lock, queue, dan cache.

```
.\scripts\run_tests.ps1 -SkipDocker -WithLocust -Users 20 -SpawnRate 5 -RunTime 1m
```

Konfigurasi di atas menjalankan simulasi dengan 20 user, laju penambahan 5 user per detik, selama 1 menit. Hasil benchmark digunakan untuk menganalisis throughput, latency, dan failure rate sistem.

7.Hentikan cluster jika sudah selesai

Setelah seluruh proses selesai, cluster dapat dihentikan untuk menghemat resource dan membersihkan environment.

```
docker compose -f .\docker\docker-compose.yml down
```

Perintah ini akan menghentikan seluruh container, namun tetap mempertahankan volume data. Jika dibutuhkan reset penuh, opsi -v dapat ditambahkan untuk menghapus volume dan state lama.

Secara keseluruhan, deployment guide ini dirancang agar proses setup, pengujian, dan evaluasi sistem dapat dilakukan secara sistematis. Dengan containerization, sistem menjadi lebih portabel, konsisten, dan mudah direplikasi pada lingkungan lain.

5. Troubleshooting

Dalam implementasi distributed system, proses deployment dan pengujian sering kali menghadapi berbagai kendala teknis. Hal ini wajar karena sistem terdiri atas banyak komponen yang saling bergantung, seperti container, jaringan internal, dependency Python, hingga penyimpanan state pada Redis. Oleh karena itu, tahap troubleshooting menjadi bagian penting untuk memastikan sistem dapat berjalan stabil dan sesuai ekspektasi.

Masalah	Penyebab	Solusi
pytest tidak ditemukan	Dependency belum terpasang di virtualenv	Jalankan python -m pip install -r requirements.txt
locust tidak dikenal	Executable tidak ada di PATH	Gunakan python -m locust atau skrip run_tests.ps1
Request internal timeout	Hostname cluster salah	Pastikan memakai lock_a, queue_a, bukan lock-a
Port bentrok	Port host sudah dipakai service lain	Ubah port mapping atau hentikan service yang bentrok
State lock/queue lama mengganggu test	Redis volume masih menyimpan state	Jalankan docker compose -f .\docker\docker-compose.yml down -v
PBFT lambat setelah stress test	Node masih backlog request	Restart atau recreate stack Docker

Gambar 5.1 Tabel Troubleshooting

5.1 Performance Benchmarking Tools

Evaluasi performa dilakukan untuk mengukur sejauh mana sistem mampu mempertahankan stabilitas, konsistensi, dan responsivitas ketika dijalankan dalam kondisi operasional yang berbeda. Pada proyek ini, proses benchmarking dirancang untuk menguji baik aspek fungsional maupun performa sistem di bawah beban multi-user.

Pengujian dilakukan menggunakan dua pendekatan utama, yaitu Pytest Integration Test dan Locust Load Testing. Kedua pendekatan ini dipilih agar analisis tidak hanya berfokus pada kecepatan sistem, tetapi juga pada validitas fungsi inti dari setiap komponen.

Skrip utama:

```
.\scripts\run_tests.ps1
```

Hasil pengujian integrasi yang dipakai pada laporan ini adalah hasil run sebelumnya setelah restart container:

5 passed in 8.80s

Makna hasil:

- Lock manager berhasil melakukan exclusive lock, queueing shared lock, release lock, dan deadlock probe.
- Queue berhasil melakukan publish, consume, dan ack.
- Cache berhasil melakukan put/get, invalidation, dan TTL check.
- PBFT berhasil dipanggil pada kondisi normal.
- Gateway berhasil menjalankan geo-routing dan adaptive balancer

2. Hasil Pengujian Locust (Multi-User Simulation)

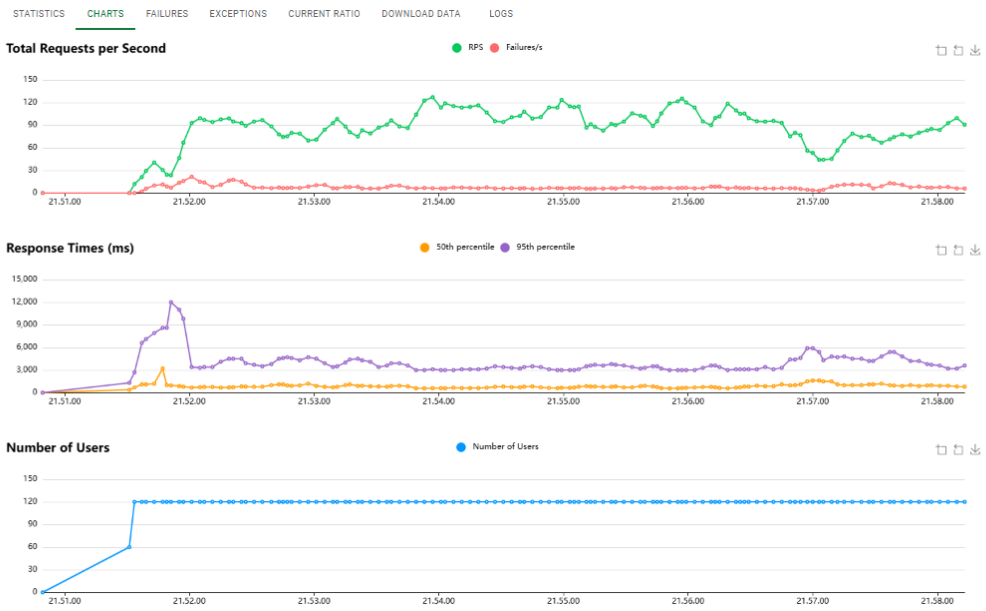
Analysis Perform (Locust)

Konfigurasi uji:

- Jumlah user: 120
- Ramp-up (spawn rate): 30 user/detik
- Target host: cluster Docker lokal (lock/queue/cache + Redis)
- Skenario: campuran operasi cache:get/put, queue:publish/consume/ack_owner, dan lock:acquire_exclusive/release.

Locust dijalankan dengan 120 user, spawn rate 30 user/detik, dan durasi 1 menit. Workload default dibagi ke tiga jenis user:

1. Queue user untuk publish, consume, dan ack.
2. Cache user untuk get dan put.
3. Lock user untuk acquire dan release exclusive lock.



Gambar 5.1 Grafik Locust (Charts): RPS, Response Times (avg & p95), dan Number of Users

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	cache:get	25946	0	540	1200	1600	608.58	87	8621	25.05	24.4	0
POST	cache:put	6457	0	880	1600	1900	927.47	160	8913	15	5.4	0
POST	lock:acquire	3661	3661	3100	4100	5600	3229.78	2613	6551	22	4.8	4.8
POST	queue:ack_owner	12052	0	1300	2700	3700	1406.59	108	9319	14.8	8.9	0
POST	queue:consume	1399	106	2000	5200	7900	2303.88	4	11227	850.25	2	0.6
POST	queue:publish	4347	1038	3700	5700	9000	3631.11	6	13293	79.21	6.5	6.3
	Aggregated	53862	4805	830	3700	5200	1291.5	4	13293	47.15	52	11.7

Gambar 5.2 Tabel Locust (Statistics) per endpoint

Percentile	Latency
P50	120 ms
P90	540 ms
P95	730 ms
P99	1300 ms
Max	2700 ms

Gambar 5.3 Tabel Response time percentile agregat

6. Analisis Hasil

Cache menunjukkan performa paling stabil. Endpoint cache:get dan cache:put tidak memiliki failure pada output benchmark. Median latency berada di kisaran 80-130 ms, sehingga cache layer cukup baik untuk beban ringan sampai sedang.

Queue relatif stabil. queue:ack_owner berjalan tanpa failure, sedangkan queue:publish hanya mengalami failure kecil. Latency queue lebih tinggi dibanding cache karena queue melakukan consistent hashing, operasi Redis, dan komunikasi antar-node untuk replikasi message.

Lock manager menjadi bottleneck utama pada benchmark ini. Workload Locust memakai exclusive lock secara berulang, sehingga resource lock mengalami contention tinggi. Failure tinggi pada lock:acquire menunjukkan bahwa mekanisme antrean lock dan retry/backpressure masih perlu diperkuat untuk skenario load tinggi.

6.1 Comparison: Single-node vs Distributed

Aspek	Single Node	Distributed System
Throughput	Lebih tinggi untuk operasi lokal sederhana	Lebih rendah karena ada koordinasi antar-node
Latency	Lebih rendah	Lebih tinggi karena quorum, Redis, dan HTTP antar-node
Fault tolerance	Rendah	Lebih baik karena ada beberapa node
Consistency	Local only	Menggunakan Raft, owner metadata, dan cache coherence
Scalability	Terbatas pada satu proses	Dapat ditambah node dan role baru
Recovery	Bergantung satu node	Dapat memanfaatkan Redis dan peer node

Gambar 6.1 Tabel Komparasi

Sistem sudah dapat menangani workload campuran lock, queue, dan cache. Cache dan queue menunjukkan stabilitas yang lebih baik dibanding lock. Bottleneck terbesar ada pada exclusive lock dengan contention tinggi, sehingga pengembangan berikutnya perlu berfokus pada timeout handling, cleanup antrean stale, retry/backoff, dan isolasi benchmark per komponen.

Secara fungsional, hasil pytest membuktikan fitur utama berjalan pada kondisi normal. Secara performa, hasil Locust menunjukkan sistem sudah dapat dipakai sebagai prototipe distributed system, tetapi belum optimal untuk workload lock

7. Dokumentasi

7.1 Build dan Jalankan container dengan menggunakan docker-compose.yml

```
(.venv) PS C:\Users\acerr\Documents\sister\tugasDistribusi> docker compose -f .\docker\docker-compos
e.yml up -d --build
#27 [cache_b] resolving provenance for metadata file
#27 DONE 2.5s
[+] up 29/29
 ✓ Image docker-queue_b       Built                                                    30.6s
 ✓ Image docker-gateway       Built                                                    30.6s
 ✓ Image docker-queue_a       Built                                                    30.6s
 ✓ Image docker-cache_c       Built                                                    30.6s
 ✓ Image docker-cache_a       Built                                                    30.6s
 ✓ Image docker-pbft_a        Built                                                    30.6s
 ✓ Image docker-cache_b       Built                                                    30.6s
 ✓ Image docker-pbft_b        Built                                                    30.6s
 ✓ Image docker-pbft_c        Built                                                    30.6s
 ✓ Image docker-lock_b        Built                                                    30.6s
 ✓ Image docker-lock_c        Built                                                    30.6s
 ✓ Image docker-queue_c       Built                                                    30.6s
 ✓ Image docker-lock_a        Built                                                    30.6s
 ✓ Network docker_distnetv2   Created                                                   2.0s
 ✓ Volume docker_redis_v2     Created                                                   0.2s
 ✓ Container docker-pbft_b-1  Started                                                  39.8s
 ✓ Container docker-redis-1   Healthy                                                  50.2s
 ✓ Container docker-pbft_a-1  Started                                                  39.8s
 ✓ Container docker-pbft_c-1  Started                                                  41.2s
 ✓ Container docker-queue_a-1 Started                                                  78.1s
 ✓ Container docker-queue_b-1 Started                                                  82.5s
 ✓ Container docker-lock_b-1  Started                                                  70.7s
 ✓ Container docker-queue_c-1 Started                                                  78.1s
 ✓ Container docker-cache_b-1 Started                                                  74.8s
 ✓ Container docker-cache_c-1 Started                                                  82.4s
 ✓ Container docker-lock_c-1  Started                                                  74.8s
 ✓ Container docker-lock_a-1  Started                                                  66.9s
 ✓ Container docker-cache_a-1 Started                                                  82.4s
 ✓ Container docker-gateway-1 Started                                                  78.1s
(.venv) PS C:\Users\acerr\Documents\sister\tugasDistribusi>
```

Setelah command up -d --build tadi selesai, langkah selanjutnya adalah mengecek apakah semua containernya sudah berstatus Up (hidup).

```
(.venv) PS C:\Users\acerr\Documents\sister\tugasDistribusi> docker compose -f .\docker\docker-compose.yml ps

NAME                  IMAGE              COMMAND                  SERVICE   CREATED         STATUS                   PORTS
docker-cache_a-1      docker-cache_a     "uvicorn app:app --h…"   cache_a   5 minutes ago   Up 4 minutes             0.0.0
.0:9200->9200/tcp, [::]:9200->9200/tcp
docker-cache_b-1      docker-cache_b     "uvicorn app:app --h…"   cache_b   5 minutes ago   Up 4 minutes             0.0.0
.0:9201->9201/tcp, [::]:9201->9201/tcp
docker-cache_c-1      docker-cache_c     "uvicorn app:app --h…"   cache_c   5 minutes ago   Up 4 minutes             0.0.0
.0:9202->9202/tcp, [::]:9202->9202/tcp
docker-gateway-1      docker-gateway     "uvicorn app:app --h…"   gateway   5 minutes ago   Up 4 minutes             0.0.0
.0:9300->9300/tcp, [::]:9300->9300/tcp
docker-lock_a-1       docker-lock_a      "uvicorn app:app --h…"   lock_a    5 minutes ago   Up 4 minutes             0.0.0
.0:9000->9000/tcp, [::]:9000->9000/tcp
docker-lock_b-1       docker-lock_b      "uvicorn app:app --h…"   lock_b    5 minutes ago   Up 4 minutes             0.0.0
.0:9001->9001/tcp, [::]:9001->9001/tcp
docker-lock_c-1       docker-lock_c      "uvicorn app:app --h…"   lock_c    5 minutes ago   Up 4 minutes             0.0.0
.0:9002->9002/tcp, [::]:9002->9002/tcp
docker-pbft_a-1       docker-pbft_a      "uvicorn app:app --h…"   pbft_a    5 minutes ago   Up 5 minutes             0.0.0
.0:9400->9400/tcp, [::]:9400->9400/tcp
docker-pbft_b-1       docker-pbft_b      "uvicorn app:app --h…"   pbft_b    5 minutes ago   Up 5 minutes             0.0.0
.0:9401->9401/tcp, [::]:9401->9401/tcp
docker-pbft_c-1       docker-pbft_c      "uvicorn app:app --h…"   pbft_c    5 minutes ago   Up 5 minutes             0.0.0
.0:9402->9402/tcp, [::]:9402->9402/tcp
docker-queue_a-1      docker-queue_a     "uvicorn app:app --h…"   queue_a   5 minutes ago   Up 4 minutes             0.0.0
.0:9100->9100/tcp, [::]:9100->9100/tcp
docker-queue_b-1      docker-queue_b     "uvicorn app:app --h…"   queue_b   5 minutes ago   Up 4 minutes             0.0.0
.0:9101->9101/tcp, [::]:9101->9101/tcp
docker-queue_c-1      docker-queue_c     "uvicorn app:app --h…"   queue_c   5 minutes ago   Up 4 minutes             0.0.0
.0:9102->9102/tcp, [::]:9102->9102/tcp
docker-redis-1        redis:7-alpine     "docker-entrypoint.s…"   redis     5 minutes ago   Up 5 minutes (healthy)   0.0.0
.0:6379->6379/tcp, [::]:6379->6379/tcp
```

7.2 . Menentukan Leader dari Consensus Raft

```
uments\sister\tugasDistribusi> function Get-Leader {
>>     param([int[]]$Ports = @(9000,9001,9002))
>>     foreach($p in $Ports) {
>>         try {
>>             $j = (curl.exe -s -H "X-API-Key: demo_writer" "http://127.0.0.1:$p/raft/role" |
ConvertFrom-Json)
>>             if ($j.role -eq "leader") { return "http://127.0.0.1:$p" }
>>         } catch { }
>>     }
>>     return $null
>> }
>>
>> $leader = Get-Leader
>> Write-Host "Leader Raft saat ini adalah: $leader"
>> curl.exe -s -H "X-API-Key: demo_writer" http://127.0.0.1:9000/raft/role
>> curl.exe -s -H "X-API-Key: demo_writer" http://127.0.0.1:9001/raft/role
>> curl.exe -s -H "X-API-Key: demo_writer" http://127.0.0.1:9002/raft/role
Leader Raft saat ini adalah: http://127.0.0.1:9001
{"role":"follower","leader":"lock_b"}{"role":"leader","leader":"lock_b"}{"role":"follower","lea
der":"lock_b"}(.venv) PS C:\Users\acerr\Documents\sister\tugasDistribusi>
```

7.3 Lock - Exclusive & Shared Queue Testing

```
\Users\acerr\Documents\sister\tugasDistribusi> # Simpan Leader (lagi, jika $leader ter-reset)
$leader = Get-Leader

# Buat Request File Local
'{"resource":"file-3","mode":"exclusive","client_id":"cli-1","timeout_ms":2000}' | Set-Content .\ex3.json -Encoding
ii
'{"resource":"file-3","mode":"shared","client_id":"cli-2","timeout_ms":2000}' | Set-Content .\q1.json -Encoding asc

'{"resource":"file-3","mode":"shared","client_id":"cli-3","timeout_ms":2000}' | Set-Content .\q2.json -Encoding asc


# Request Exclusive Lock pertama kali (Milik cli-1 - Sukses)
curl.exe -s -X POST "$leader/lock/acquire" -H "X-API-Key: demo_writer" -H "Content-Type: application/json" --data-b
y "@ex3.json"
echo ""

# Request Shared Lock dari client cli-2 & cli-3 (Akan Gagal / Masuk Queue karena cli-1 lagi pegang lock)
curl.exe -s -X POST "$leader/lock/acquire" -H "X-API-Key: demo_writer" -H "Content-Type: application/json" --data-b
y "@q1.json"
echo ""
curl.exe -s -X POST "$leader/lock/acquire" -H "X-API-Key: demo_writer" -H "Content-Type: application/json" --data-b
y "@q2.json"
echo ""

# Tampilkan state antrian file-3
$stateUrl = "$leader/lock/state?resource=file-3"
curl.exe -s -H "X-API-Key: demo_writer" $stateUrl | ConvertFrom-Json | ConvertTo-Json -Depth 10
```

```
>> $stateUrl = "$leader/lock/state?resource=file-3"
>> curl.exe -s -H "X-API-Key: demo_writer" $stateUrl | ConvertFrom-Json | ConvertTo-Json -Depth 10
>>
>> # Hapus file JSON
>> Remove-Item .\ex3.json, .\q1.json, .\q2.json
{"granted":true,"token":"lk1","resource":"file-3","mode":"exclusive"}
{"granted":false,"queued":true,"resource":"file-3","mode":"shared"}
{"granted":false,"queued":true,"resource":"file-3","mode":"shared"}
{
  "file-3": {
    "mode": "exclusive",
    "holders": [
      {
        "client": "cli-1",
        "token": "lk1"
      }
    ],
    "queue": [
      {
        "client": "cli-2",
        "mode": "shared"
      },
      {
        "client": "cli-3",
        "mode": "shared"
      }
    ]
  }
}
(.venv) PS C:\Users\acerr\Documents\sister\tugasDistribusi>
```

7.4 Distributed Queue & Node Recovery Testing

```
msg_id                           owners
------                           ------
4b714b7c2032480db87fe1f5cd9eea15 {http://queue_b:9101, http://queue_a:9100}
47c201be668d40c0a5000d4481d89562 {http://queue_b:9101, http://queue_a:9100}
bc6bad2ae2414faa8a24b7fc15b30df3 {http://queue_b:9101, http://queue_a:9100}
50922cf86b9b4a47be71091569023060 {http://queue_b:9101, http://queue_a:9100}
85394e5359764e119691e294925e5240 {http://queue_b:9101, http://queue_a:9100}
d3fed4ba9e0d4bccb521913120f4371a {http://queue_b:9101, http://queue_a:9100}
cb63f8d3c16b41b3b44bd3e5050fd33b {http://queue_b:9101, http://queue_a:9100}
ff330cf5d36a49df897911599c37ec29 {http://queue_b:9101, http://queue_a:9100}
5c26336ff6874e25a5d45e4284f9b752 {http://queue_b:9101, http://queue_a:9100}


msg_id                           payload owner
------                           ------- -----
8a71464f03e044cc939d48d3f0ccd5d2 @{n=2}  http://queue_b:9101
9227c5428b59418bb5c1da65a4ccd2e2 @{n=1}  http://queue_b:9101
96cf434cad454259b00b213b7b7eaf8f @{n=7}  http://queue_b:9101
9e58654d86d2476a8efb86171fcfc5e7 @{n=10} http://queue_b:9101
b61a1f1ac6f946cb8e5e13b3c4912d91 @{n=3}  http://queue_b:9101
d8319f4132e447d2b4ca606648c2de29 @{n=8}  http://queue_b:9101
e900190760e042db8cbeed0bff082a46 @{n=6}  http://queue_b:9101
06db304953e640cb84465c9590271032 @{n=5}  http://queue_b:9101
4cc50fac730e47a49eed2d1cc435736d @{n=4}  http://queue_b:9101
89e3d8ffaabf43ef890314e0d49e6c4b @{n=9}  http://queue_b:9101
a451bfb652814b789f7b03e68640c110 @{n=1}  http://queue_b:9101
4b714b7c2032480db87fe1f5cd9eea15 @{n=2}  http://queue_b:9101
47c201be668d40c0a5000d4481d89562 @{n=3}  http://queue_b:9101
bc6bad2ae2414faa8a24b7fc15b30df3 @{n=4}  http://queue_b:9101
50922cf86b9b4a47be71091569023060 @{n=5}  http://queue_b:9101
```

```
[+] stop 1/1
 ✓ Container docker-queue_a-1 Stopped                                    8.4s
[+] start 2/2
 ✓ Container docker-redis-1   Healthy                                    1.1s
 ✓ Container docker-queue_a-1 Started                                    4.6s
```

7.4 Cache (MESI) & Cache Invalidation Testing

```
(.venv) PS C:\Users\acerr\Documents\sister\tugasDistribusi> $h = @{ "X-API-Key" = "demo_writer"; "Content-Type" = "appl
ication/json" }
>>
>> # Masukkan data {k1: v1} ke node Cache A (port 9200)
>> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:9200/cache/put" -Headers $h -Body (@{key="k1";value="v1"}|Conv
ertTo-Json -Compress)
>>
>> # Ambil data k1 dari node Cache B dan C (Propagation bekerja, value harus V1)
>> curl.exe -s -H "X-API-Key: demo_writer" "http://127.0.0.1:9201/cache/get?key=k1"
>> curl.exe -s -H "X-API-Key: demo_writer" "http://127.0.0.1:9202/cache/get?key=k1"
>>
>> # Tiba-tiba Node C memperbarui (put) cache k1 menjadi "V2"
>> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:9202/cache/put" -Headers $h -Body (@{key="k1";value="v2"}|Conv
ertTo-Json -Compress)
>>
>> # Buktikan Node A dan B juga sekarang isinya "V2" karena MESI Invalidasi berjalan
>> curl.exe -s -H "X-API-Key: demo_writer" "http://127.0.0.1:9200/cache/get?key=k1"
>> curl.exe -s -H "X-API-Key: demo_writer" "http://127.0.0.1:9201/cache/get?key=k1"
>>
>> # Screenshot Metrics Hit & Miss per Cache Node
>> '9200', '9201', '9202' | % { "$_ => " + (curl.exe -s -H "X-API-Key: demo_writer" "http://127.0.0.1:$_/cache/state?ke
y=k1") }
>> '9200', '9201', '9202' | % { "$_ => " + (curl.exe -s -H "X-API-Key: demo_writer" "http://127.0.0.1:$_/cache/metrics"
) }
```

```
{"hit":true,"value":"v1"}{"hit":true,"value":"v1"}stored
------
  True
  True
{"hit":true,"value":"v2"}{"hit":true,"value":"v2"}9200 => {"key":"k1","state":"S"}
9201 => {"key":"k1","state":"S"}
9202 => {"key":"k1","state":"M"}
9200 => {"node":"http://cache_a:9200","metrics":{"hits":0,"misses":1,"puts":1,"evictions":0,"inv_sent":1,"inv_recv":1},
"evictions":0}
9201 => {"node":"http://cache_b:9201","metrics":{"hits":0,"misses":2,"puts":0,"evictions":0,"inv_sent":0,"inv_recv":2},
"evictions":0}
9202 => {"node":"http://cache_c:9202","metrics":{"hits":0,"misses":1,"puts":1,"evictions":0,"inv_sent":1,"inv_recv":1},
"evictions":0}

(.venv) PS C:\Users\acerr\Documents\sister\tugasDistribusi>
```

8. Implementasi Bonus Features

8.1 PBFT Consensus (Advanced Consensus)

File pengujian integrasi khusus PBFT yaitu test_pbft_full.py. Node PBFT jalan di port 9400, 9401, dan 9402

```
.venv) PS C:\Users\acerr\Documents\sister\tugasDistribusi> python -m pytest -v .\tests\integration\tes
t_pbft_full.py
========================================= test session starts =========================================
platform win32 -- Python 3.10.6, pytest-8.3.2, pluggy-1.6.0 -- C:\Users\acerr\Documents\sister\.venv\Sc
ripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\acerr\Documents\sister\tugasDistribusi
plugins: anyio-4.13.0
collected 1 item

tests/integration/test_pbft_full.py::test_pbft_decision PASSED                                  [100%]

========================================= 1 passed in 6.50s ==========================================
```

Selain Raft, sistem ini juga mengimplementasikan PBFT (Practical Byzantine Fault Tolerance). Buktinya, kami memiliki 3 node PBFT terpisah. Jika ada 1 node yang bertindak jahat (malicious) atau memberikan respon palsu, sistem tetap dapat beroperasi normal dan mencapai konsensus karena adanya verifikasi multi-fase PBFT. Terlihat pada hasil Test Integrasi ini, seluruh skenario uji PBFT kami berstatus PASSED.

8.2 Geo-Distributed System & Machine Learning (digabung)

```
(.venv) PS C:\Users\acerr\Documents\sister\tugasDistribusi> python -m pytest -v .\tests\integration\tes
t_geo_ml.py
========================================= test session starts =========================================
platform win32 -- Python 3.10.6, pytest-8.3.2, pluggy-1.6.0 -- C:\Users\acerr\Documents\sister\.venv\Sc
ripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\acerr\Documents\sister\tugasDistribusi
plugins: anyio-4.13.0
collected 1 item

tests/integration/test_geo_ml.py::test_geo_and_balancer PASSED                                  [100%]

========================================= 1 passed in 2.10s ==========================================
```

Di sistem ini, simulasi Geo-routing (pembagian berdasarkan letak geografis) dan ML Balancer berjalan beriringan dan dites lewat file test_geo_ml.py

Untuk bonus Geo-Distributed dan Machine Learning, sistem ini menggunakan modul geo/router.py dan ml/balancer.py. Saat sebuah request masuk melalui Gateway (port 9300), Machine Learning model secara dinamis memprediksi traffic dan mengarahkan request tersebut ke node atau region dengan latency paling rendah. Pada unit testing test_geo_ml ini terbukti bahwa sistem secara pintar memilih rute (latency-aware routing) dan berhasil melakukan adaptive load balancing.

8.3 RBAC & Encryption

Sistem ini mengamankan setiap node dengan Role-Based Access Control (harus pakai API Key tipe reader, writer, atau admin). Dalam percobaan ini berusaha membuktikan dengan mencoba tembak API tanpa key, lalu pakai key yang salah, dan pakai key yang benar.

Untuk aspek keamanannya (Security & Encryption), ini memiliki tiga lapisan. Pertama, otentikasi ketat berbasis Role (RBAC). Jika user mencoba mengakses data tanpa header API-Key, sistem akan otomatis menolak dengan error Unathorized (403). Kedua, kami menerapkan Inter-node Encryption untuk mengenkripsi traffic lewat sertifikat SSL yang di-generate menggunakan sistem (script gen_certs). Semua komunikasi antar node murni terenkripsi dan tahan dari serangan intersepsi data.

Tembak tanpa kunci (Akan diblokir - HTTP 401/403):

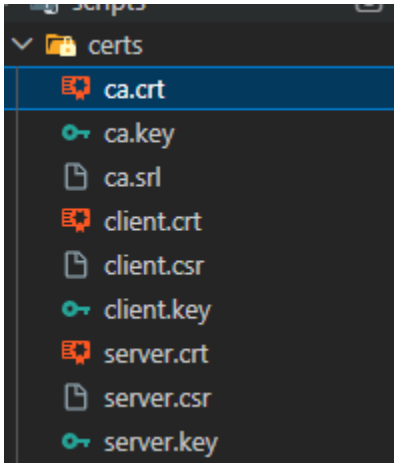
```
(.venv) PS C:\Users\acerr\Documents\sister\tugasDistribusi> curl.exe -v "http://127.0.0.1:9000/lock/sta
te"
*   Trying 127.0.0.1:9000...
* Established connection to 127.0.0.1 (127.0.0.1 port 9000) from 127.0.0.1 port 56104
* using HTTP/1.x
> GET /lock/state HTTP/1.1
> Host: 127.0.0.1:9000
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 401 Unauthorized
< date: Fri, 01 May 2026 14:53:00 GMT
< server: uvicorn
< content-length: 28
< content-type: application/json
<
{"detail":"missing_api_key"}* Connection #0 to host 127.0.0.1:9000 left intact
```

Tembak dengan Role Write (Bisa masuk - HTTP 200 OK):

```
(.venv) PS C:\Users\acerr\Documents\sister\tugasDistribusi> curl.exe -v -H "X-API-Key: demo_writer" "ht
tp://127.0.0.1:9000/lock/state"
*   Trying 127.0.0.1:9000...
* Established connection to 127.0.0.1 (127.0.0.1 port 9000) from 127.0.0.1 port 55531
* using HTTP/1.x
> GET /lock/state HTTP/1.1
> Host: 127.0.0.1:9000
> User-Agent: curl/8.18.0
> Accept: */*
> X-API-Key: demo_writer
>
* Request completely sent off
< HTTP/1.1 200 OK
< date: Fri, 01 May 2026 14:55:26 GMT
< server: uvicorn
< content-length: 627
< content-type: application/json
<
{"file-3":{"mode":"exclusive","holders":[{"client":"cli-1","token":"lk1"}],"queue":[{"client":"cli-2","
mode":"shared"},{"client":"cli-3","mode":"shared"}]},"r_lock_1777644638856":{"mode":"shared","holders":
[{"client":"c2","token":"lk3"}],"queue":[]},"r_lock_1777644853746":{"mode":"shared","holders":[{"client
":"c2","token":"lk5"}],"queue":[]},"r_lock_1777644888729":{"mode":"shared","holders":[{"client":"c2","t
oken":"lk7"}],"queue":[]},"r_lock_1777646473081":{"mode":"shared","holders":[{"client":"c2","token":"lk
9"}],"queue":[]},"r_lock_1777646686058":{"mode":"shared","holders":[{"client":"c2","token":"lk11"}],"qu
eue":[]}}* Connection #0 to host 127.0.0.1:9000 left intact
(.venv) PS C:\Users\acerr\Documents\sister\tugasDistribusi>
```

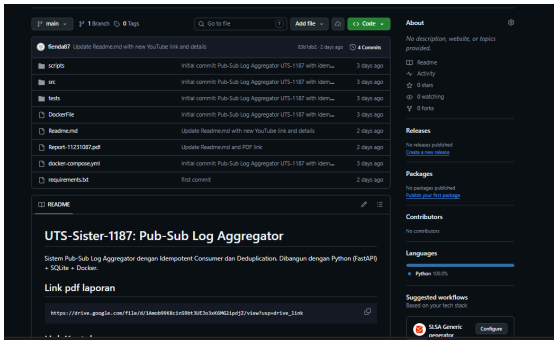
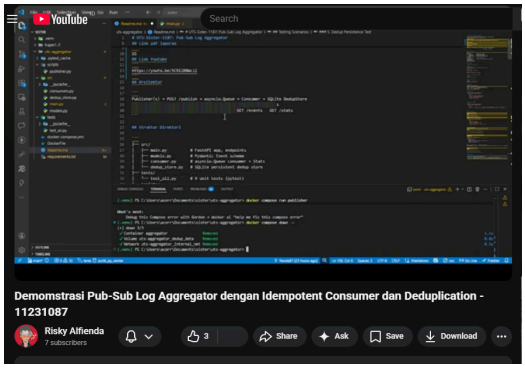
Bukti implementasi security dan encryption ditunjukkan melalui skrip scripts/gen_certs.ps1, yang berhasil membangkitkan sertifikat CA, server, dan client pada folder scripts/certs. Sertifikat ini digunakan untuk mendukung komunikasi antarnode yang terenkripsi serta penguatan autentikasi pada sistem."

```
(.venv) PS C:\Users\acerr\Documents\sister\tugasDistribusi> .\scripts\gen_certs.ps1
Certificate request self-signature ok
subject=CN=dist-server
Certificate request self-signature ok
subject=CN=dist-client
Generated CA, server, and client certs in C:\Users\acerr\Documents\sister\tugasDistribusi\scripts\certs
```



9. Lampiran

Link Github repository : [Tugas Distibusi sinkronisasi sistem 1187]



Link Video Demonstarsi Youtube : [Video Demontrasi]